



JC4002 Knowledge Representation

Day 2: Resolution

Dr Ferdian Jovan

ferdian.jovan@abdn.ac.uk

Dr Aiden Durrant

aiden.durrant@abdn.ac.uk

Dr Jiangtian Nie

jiangtian.nie@abdn.ac.uk

Day 2

Resolution

1. **Automated Reasoning**
2. Automated Theorem Provers
3. SAT Solvers

Automated Reasoning

- A computer program capable of reasoning
- Given a set of assumptions and a query, an automated reasoning system should be able to make logical inferences towards that query automatically
- Is creating an automated procedure for this hard?

Automated Reasoning

- A computer program capable of reasoning
- Given a set of assumptions and a query, an automated reasoning system should be able to make logical inferences towards that query automatically
- Is creating an automated procedure for this hard? (spoiler: yes)

Deductive Reasoning Procedure

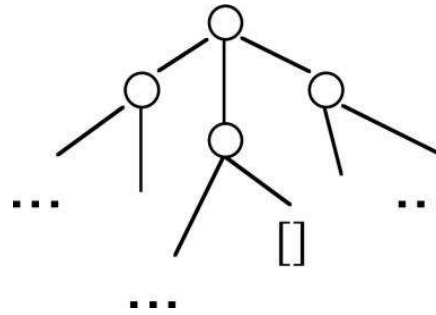
- What should our automated deductive reasoning procedure do?
 - given a knowledge base KB and a sentence α , determine if $KB \models \alpha$ **or**
 - given an open $\alpha[x_1, x_2, \dots, x_n]$, find $t_1, t_2, \dots, t_n$ such that $KB \models \alpha[t_1, t_2, \dots, t_n]$

- How can we do that?

When KB is finite $\{\alpha_1, \alpha_2, \dots, \alpha_k\}$, then $KB \models \alpha$

- iff $\models [(\alpha_1, \alpha_2, \dots, \alpha_k) \rightarrow \alpha]$ (validity)
- iff $KB \cup \{\neg\alpha\}$ is unsatisfiable (satisfiability)
- iff $KB \cup \{\neg\alpha\} \models \text{false}$ (entailing false)

Undecidability



- ❑ For example, First-Order Logic is very expressive, it can even be used as a full programming language
- ❑ As such, it has loops, and the same principle of decidability applies here, there is no way to automatically detect when a program is looping
- ❑ Resolution on the other hand is complete if we are dealing with unsatisfiable clauses
- ❑ A breadth-first search is guaranteed to find an empty clause (represented by $[]$)
- ❑ Search may not terminate on satisfiable clauses (infinite branches)

Resolution is Hard

- ❑ First-order resolution is not guaranteed to terminate
 - But if we translate it to propositional logic then there is some hope (may still require exponential time on some problems)
 - This can be done with Herbrand's Theorem
 - Herbrand's Theorem (informally) states that “if a FOL formula is valid, then there exists a finite set of ground instances of the formula's clauses (where the variables are replaced by terms from the Herbrand universe) that is propositionally valid.”
- ❑ This problem is not limited to resolution
- ❑ No easier than any other complex computational task
- ❑ Any search task where what is searched can be verified in polynomial time can be recast as a satisfiability problem

Day 2

Resolution

1. Automated Reasoning
- 2. Automated Theorem Provers**
3. SAT Solvers

Automated Theorem Prover

- ☐ Given a proof system, assumptions, and a goal to prove, an automated theorem prover will show steps between the assumptions and the goal
- ☐ Uses a verifier to check for proofs
- ☐ Extremely useful for proving mathematical theorems
- ☐ Also used in integrated circuit design
- ☐ Formal verification

Automated Theorem Prover - Examples

- ❑ Prover9: <https://www.cs.unm.edu/~mccune/prover9/>
 - Not actively maintained
 - Resolution automated theorem prover for **first-order and equational logic**
 - Successor of Otter (the first widely distributed, high-performance theorem prover for first-order logic)
- ❑ Vampire: <https://vprover.github.io/>
 - Constantly updated
 - Very fast (won many awards)
 - Main focus is proving theorems in **first-order logic**
 - Can also be used for many other things (including higher-order logic)

Day 2

Resolution

1. Automated Reasoning
2. Automated Theorem Provers
3. **SAT Solvers**

SAT Solver

- ❑ Look for a satisfying interpretation for clauses that are satisfiable
- ❑ Related to Constraint Satisfaction Problems (CSP)
- ❑ Solves constraints written in **propositional logic**
- ❑ Need Herbrand's Theorem to transform FOL to PL
- ❑ Given a formula over Boolean variables, e.g. $(x \vee y) \wedge (x \vee \neg y)$, a SAT solver outputs if the formula is satisfiable (there are possible values of x and y which make the formula true) or not (there are no such values of x and y).
- ❑ Heavy work on heuristics and program optimisations to work efficiently
- ❑ Can also be used for planning and plan recognition

SAT Solver - Examples

❑ SAT competition: <http://satcompetition.org/>

- Identify new challenging and complex benchmarks with every competition
- Promote new state-of-the-art SAT solvers

❑ Microsoft Z3 SMT solver: <https://github.com/Z3Prover/z3>

- Satisfiability Modulo Theories (SMT) problem is a decision problem for logical formulas with respect to combinations of background theories such as arithmetic, bit-vectors, arrays, and uninterpreted functions
- Besides FOL, also covers predicate logic with quantifiers
- Constantly updated

Summary

- Discussed automated reasoning for resolution of KBs
- Presented required properties in a high-level deductive reasoning procedure
- Learned about two types of tools for automated reasoning (theorem provers and SAT solvers), and saw some examples of tools for each
- What if we want to produce entailments of a KB to decide on an action?
 - If we can wait, theorem proving
 - Otherwise, use less expressive languages (such as **Horn clauses – Next class**)